

Bases - Labo Optimización 2025-1C

Query 1

```
-- Consulta A  
SELECT * FROM Sales.SalesOrderDetail  
WHERE ProductID = 897;
```

```
-- Consulta B  
DECLARE @prodId INT = 897;  
SELECT * FROM Sales.SalesOrderDetail  
WHERE ProductID = @prodId;
```

La Consulta A y la Consulta B utilizan el mismo valor, pero en muchos casos el optimizador genera un plan distinto para cada una. ¿Por qué?

- A. Porque las variables bloquean el uso de índices.
- B. Porque las consultas con variables no pueden acceder al índice clúster.
- C. Porque al usar una variable, el optimizador no puede estimar correctamente la selectividad y genera un plan más genérico.
- D. Porque SQL Server recompila la consulta cada vez que se usa una variable.

Query 2

```
-- Consulta A  
SELECT * FROM Sales.SalesOrderDetail  
WHERE ProductID = 870 OR ProductID = 897;
```

```
-- Consulta B  
SELECT * FROM Sales.SalesOrderDetail  
WHERE ProductID = 870  
UNION ALL  
SELECT * FROM Sales.SalesOrderDetail  
WHERE ProductID = 897;
```

¿Por qué el optimizador elige un plan diferente para cada consulta?

- A. Porque UNION ALL fuerza el uso de índices.
- B. Porque al usar UNION ALL, el optimizador puede aplicar Seek sobre condiciones simples y combinar los resultados.
- C. Porque el operador OR siempre ignora los índices en SQL Server.
- D. Porque el optimizador solo puede paralelizar consultas con UNION ALL.

Query 3

```
DECLARE @productId INT = 897;
```

```
SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = @productId  
OPTION (OPTIMIZE FOR (@productId = 897));
```

```
SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = @productId  
OPTION (OPTIMIZE FOR (@productId = 870));
```

Las dos consultas utilizan el mismo valor de ejecución (@productId = 897).

¿Por qué los planes son diferentes?

- A. OPTIMIZE FOR solo afecta consultas con parámetros externos, no variables locales.
- B. La hint OPTIMIZE FOR obliga a usar un índice específico.
- C. El optimizador siempre ignora OPTIMIZE FOR y usa el valor real.
- D. El optimizador genera el plan asumiendo la selectividad del valor indicado en la opción OPTIMIZE FOR, sin importar el valor real de ejecución.

Query 4

```
-- Consulta A
SELECT *
FROM Sales.SalesOrderDetail d
JOIN Production.Product p
  ON d.ProductID = p.ProductID
WHERE p.ProductID = 870;
```

```
-- Consulta B
SELECT *
FROM Sales.SalesOrderDetail d
JOIN Production.Product p
  ON d.ProductID = p.ProductID
WHERE p.Color = 'Black';
```

Las dos consultas ejecutan un JOIN entre SalesOrderDetail y Product, pero generan planes de ejecución con estrategias de junta diferentes: una usa Nested Loops y la otra Hash Match.

¿Cuál es la causa más probable de esta diferencia?

- A. Porque Hash Match se usa solamente cuando hay WHERE con cadenas de texto.
- B. El optimizador utiliza Hash Match cuando hay columnas de tipo texto en cualquier parte del WHERE, ya que considera que los tipos nvarchar y varchar son incompatibles con Nested Loops en joins.
- C. SQL Server optimiza automáticamente los JOIN para que usen Hash Match cuando hay condiciones de filtrado en la tabla secundaria, independientemente del tamaño del conjunto de datos o la selectividad del filtro.
- D. La Consulta A tiene un valor muy selectivo (ProductID = 870), por lo que el optimizador elige Nested Loops, mientras que la B filtra por una columna menos selectiva (Color = 'Black'), lo que favorece el uso de Hash Match.
- E. La Consulta A permite el uso de índices en ambas tablas, mientras que en la Consulta B no se puede utilizar ningún índice sobre la columna Color, lo que obliga a un cambio en el tipo de JOIN.

Query 5

```
-- Consulta A
SELECT * FROM Sales.SalesOrderDetail d
JOIN Production.Product p ON d.ProductID = p.ProductID;
```

```
-- Consulta B
SELECT * FROM Sales.SalesOrderDetail d
JOIN Production.Product p ON d.ProductID = p.ProductID
WHERE p.Name LIKE 'A%';
```

Ambas consultas generan el mismo plan de ejecución, a pesar de que la Consulta B tiene una cláusula WHERE.

¿Por qué el optimizador puede generar el mismo plan?

- A. Porque el filtro `p.Name LIKE 'A%'` es poco selectivo y no cambia sustancialmente el costo estimado del plan.
- B. Porque las dos consultas son equivalentes en lógica.
- C. Porque `LIKE` nunca se considera para optimización.
- D. Porque el optimizador ignora filtros en columnas de tipo texto.